Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

# COMPTE RENDU

Cloud Foundations & Virtualization — TP4 Partie 2 — Docker : déployer un conteneur dans Azure (Container Instances et Container Apps)
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)


**Étudiant :** HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan


**Filière :** Cybersécurité

**Année :** 2026/2027

**Enseignant :** M. KARRAKCHOU & M. SOUIDI

**Date :** 21 octobre 2026

# Table des matières

**Cadre du TP** **2**

**1 Exécuter le classifieur Custom Vision en local** **3**
1.1 Partie I.1 — Contenu de l'archive . . . 3
1.2 Partie I.2 — Le Dockerfile fourni . . . 4
1.3 Partie I.3 — Construction de l'image . . . 4
1.4 Partie I.4 — Exécution du conteneur en local . . . 5
1.5 Partie I.5 — Appel de l'API . . . 6

**2 Publier l'image sur Docker Hub** **7**

**3 Exécuter le conteneur dans Azure : Container Instance** **9**
3.1 Partie III.1 — Groupe de ressources . . . 9
3.2 Partie III.2 — Création de la Container Instance . . . 9
3.3 Partie III.3 — Déploiement . . . 10
3.4 Partie III.4 — Adresse IP publique . . . 10
3.5 Partie III.5 — Appel de l'API déployée . . . 11

**4 Rendre le conteneur extensible : Container App** **12**
4.1 Partie IV.1 — Création de la Container App . . . 12
4.2 Partie IV.2 — Déploiement . . . 13
4.3 Partie IV.3 — URL de l'application . . . 13
4.4 Partie IV.4 — Appel de l'API . . . 14

**5 Modifier la règle de mise à l'échelle** **14**
5.1 Partie V.1 — Principe . . . 14
5.2 Partie V.2 — Nouvelle règle . . . 14
5.3 Partie V.3 — Lecture des paramètres . . . 15

**6 Devoir : exécuter un conteneur dans Azure** **16**
6.1 Les étapes . . . 16
6.2 Container Instance ou Container App . . . 17
6.3 Suppression des ressources . . . 17

**7 Bilan des éléments créés** **18**

**8 Difficultés rencontrées** **18**

# Cadre du TP

| **Niveau** | **Cours** | **Plateformes** |
|---|---|---|
| 4e CY | Cloud Foundations | Docker Engine / Docker Hub / Microsoft Azure (abonnement *Azure for Students*) : Container Instances et Container Apps |
| **Mode** | **Approche** | **Livrable** |
| Groupe de 2 à 3 | Exécuter, publier puis déployer dans Azure un classifieur d'images Custom Vision conteneurisé | Compte rendu + captures d'écran |

TABLE 1 – Cadre général du TP

**Objectif.** Dans la Partie 1, les conteneurs ont été exécutés, construits et partagés sur une seule machine. Cette seconde partie les emmène dans le **cloud** : on exécute en local l'image d'un **classifieur de bananes** (mûres / pas mûres) exportée depuis le service Azure *Custom Vision*, on la **publie** sur Docker Hub, puis on la **déploie dans Azure** de deux façons : comme **Container Instance** (un conteneur isolé avec une adresse IP publique) et comme **Container App** avec une règle de **mise à l'échelle automatique** selon le nombre de requêtes HTTP. Dans chaque cas, le service est appelé par un petit programme Python.

**Fil conducteur du TP :**

**Image Custom Vision → `docker build` / `docker run` local → Appel de l'API en Python → `docker push` vers Docker Hub → Azure Container Instance → Azure Container App → Règle de mise à l'échelle**

**Environnement.** Docker Engine sous Linux pour les parties locales, compte **Docker Hub** `ygghest` pour le registre, et portail **Microsoft Azure** (abonnement *Azure for Students*, région **Spain Central**) pour les parties III à V. Les ressources Azure sont regroupées dans le groupe de ressources `tp_docker`. Les fichiers fournis sur Moodle sont l'archive `bananaClassifierDocker.zip` (modèle exporté et `Dockerfile`) et l'archive `bananaDataset.zip` (images ayant servi à l'entraînement).

# 1 Exécuter le classifieur Custom Vision en local

*Partie I du TP : exploration de l'archive fournie, construction de l'image, exécution d'un conteneur en local et appel de son API avec un programme Python.*

## 1.1 Partie I.1 — Contenu de l'archive

**Custom Vision** est un service Azure qui permet d'entraîner un classifieur d'images à partir d'exemples. Le modèle du TP distingue les bananes `ripe` (mûres) des bananes `unripe` (pas mûres) ; il a été exporté sous forme d'un projet Docker dont voici le contenu.

Listing 1 – Contenu de l'archive `bananaClassifierDocker.zip`

```
Dockerfile
README.txt
LICENSE
app/
    app.py                      # serveur web Flask (les routes de l'
       API)
    predict.py                  # chargement du modèle et prédiction
    model.tflite                # modèle TensorFlow Lite exporté
    labels.txt                  # étiquettes : ripe, unripe
    metadata_properties.json    # paramètres de prétraitement
    cvexport.manifest
azureml/                        # fichiers pour AzureML (non utilisés
    ici)
```

Table 2: Fichiers principaux de l'archive

| **Fichier** | **Rôle** |
|---|---|
| `model.tflite` | Réseau de neurones entraîné, au format TensorFlow Lite, exécuté par `tflite-runtime` |
| `labels.txt` | Une étiquette par ligne (`ripe`, `unripe`), dans l'ordre des sorties du modèle |
| `predict.py` | Prétraite l'image (redimensionnement à 256 px sur le plus petit côté, puis recadrage central de 224×224 px), lance le modèle et renvoie les probabilités sous forme de JSON |
| `app.py` | Serveur Flask écoutant sur le port 80 de l'intérieur du conteneur : il expose les routes `/image` et `/url` |
| `README.txt` | Mode d'emploi : commandes de construction, d'exécution et d'appel de l'API |

L'archive `bananaDataset.zip` contient les images d'entraînement : **53 images** au total, rangées dans deux dossiers, `ripe` (24 images) et `unripe` (29 images). Elles ne sont pas nécessaires à l'exécution du conteneur ; elles servent seulement à voir à quoi ressemblent les deux catégories et à fournir des images de test.

## 1.2 Partie I.2 — Le Dockerfile fourni

Listing 2 – Dockerfile exporté par Custom Vision

```
FROM python:3.9-slim-bookworm

RUN pip install --no-cache-dir "flask<3" "pillow<11" "numpy<2"
   tflite-runtime~=2.13.0

COPY app /app
EXPOSE 80
WORKDIR /app

CMD python -u app.py
```

Table 3: Instructions du Dockerfile du classifieur

| **Instruction** | **Rôle** |
|---|---|
| `FROM python:3.9-slim-bookworm` | Image de base : Python 3.9 sur Debian *bookworm* dans sa variante allégée (*slim*) |
| `RUN pip install -no-cache-dir ...` | Installe pendant la construction Flask (serveur web), Pillow (lecture des images), NumPy (calcul) et `tflite-runtime` (exécution du modèle). Les versions sont bornées (`flask<3`, `numpy<2`...) pour éviter les incompatibilités, et `-no-cache-dir` évite de garder le cache de pip dans l'image |
| `COPY app /app` | Copie le dossier `app` de l'hôte (code, modèle, étiquettes) vers `/app` dans l'image |
| `EXPOSE 80` | Documente que l'application écoute sur le port 80 ; cela ne publie pas le port, c'est le rôle de `-p` à l'exécution |
| `WORKDIR /app` | Dossier de travail : `model.tflite` et `labels.txt` sont lus avec des chemins relatifs |
| `CMD python -u app.py` | Commande lancée au démarrage du conteneur ; `-u` désactive la mise en mémoire tampon de la sortie pour que les journaux apparaissent immédiatement |

## 1.3 Partie I.3 — Construction de l'image

Suivant le `README.txt`, l'image est construite depuis le dossier qui contient le `Dockerfile`.

Listing 3 – Construction de l'image du classifieur

```
docker build -t banana-classifier .
```




FIGURE 1 – `docker build -t banana-classifier .` : construction terminée en 31,3 s. L'image de base `python:3.9-slim-bookworm` est téléchargée, puis les étapes `RUN pip install` (15,8 s), `COPY` et `WORKDIR` sont exécutées, et l'image est nommée `docker.io/library/banana-classifier:latest`.

La sortie est celle du constructeur BuildKit : quatre étapes numérotées `[1/4]` à `[4/4]`, correspondant aux instructions du Dockerfile qui modifient l'image (`EXPOSE` et `CMD` ne sont que des métadonnées). L'étape la plus longue est l'installation des bibliothèques Python. L'option `-t banana-classifier` donne un nom à l'image et le `.` final désigne le dossier courant comme contexte de construction.

Le message jaune final (*JSONArgsRecommended*) est un simple avertissement et non une erreur : l'instruction `CMD python -u app.py` est écrite en *forme shell*, alors que la forme JSON (`CMD ["python", "-u", "app.py"]`) est recommandée, car elle lance directement le processus sans passer par un shell et lui transmet correctement les signaux d'arrêt (par exemple lors d'un `docker stop`). L'image fonctionne telle quelle.

## 1.4 Partie I.4 — Exécution du conteneur en local

Listing 4 – Exécution du classifieur en local

```
1 docker run -d --rm -p 80:80 banana-classifier
```

| Élément | Signification |
|---|---|
| `docker run` | Crée un conteneur à partir d'une image et le démarre |
| `-d` | Mode détaché : le conteneur tourne en arrière-plan et le terminal reste libre |

| Élément | Signification |
|---|---|
| `-rm` | Supprime automatiquement le conteneur dès qu'il s'arrête, ce qui évite d'accumuler des conteneurs arrêtés |
| `-p 80:80` | Redirection de port `hôte:conteneur` : le port 80 de la machine est relié au port 80 du conteneur, où écoute Flask |
| `banana-classifier` | Image construite à l'étape précédente |

TABLE 4: Options de la commande `docker run` de la Partie I

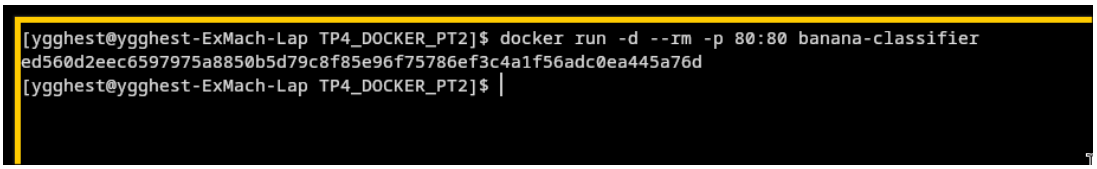



FIGURE 2 – `docker run -d -rm -p 80:80 banana-classifier` : Docker affiche l'identifiant complet du conteneur créé (`ed560d2eec65...`).

Le `README.txt` propose `-p 127.0.0.1:80:80`, qui n'ouvre le port que sur l'interface locale de la machine. La commande `-p 80:80` utilisée ici publie le port sur toutes les interfaces : le service est donc aussi joignable depuis le réseau. Pour un simple test en local, la forme du `README.txt` est la plus prudente.

## 1.5 Partie I.5 — Appel de l'API

D'après le `README.txt` et le fichier `app.py`, le conteneur expose deux routes en `POST` :

TABLE 5: Routes de l'API du classifieur

| Route | Contenu de la requête | Usage |
|---|---|---|
| `/image` | `multipart/form-data` avec la clé `imageData` | Envoi d'un fichier image depuis un formulaire |
| `/image` | `application/octet-stream` | Envoi des octets bruts d'une image |
| `/url` | JSON : `{ "url": "<adresse de l'image>" }` | Le conteneur télécharge lui-même l'image à partir de son adresse Internet |

Exemple donné par le `README.txt` pour tester en ligne de commande :

Listing 5 – Appel avec curl (extrait du README.txt)

```
curl -X POST http://127.0.0.1/image -H "Content-Type: application/
    octet-stream" --data-binary @some_file_name.jpg
```

Le programme Python du TP suit le modèle du sujet (module `requests`, requête `POST` avec en-tête personnalisé). Il envoie ici une image locale, `banana.jpg`, aux octets bruts vers la route `/image` : seule la variable `api_url` change d'un déploiement à l'autre.

Listing 6 – Programme d'appel `test_banana.py`

```
import requests
import json

api_url = "http://68.221.136.111/image"

# On ouvre l'image locale en mode binaire ('rb')
image_path = "banana.jpg"

with open(image_path, "rb") as image_file:
    image_data = image_file.read()

headers = {
    "Content-Type": "application/octet-stream"
}

# On envoie les octets bruts de l'image
response = requests.post(api_url, headers=headers, data=image_data)

print("Status Code:", response.status_code)
print("Response Text:", response.text)

# Si le statut est 200, on affiche les prédictions
if response.status_code == 200:
    result = response.json()
    for prediction in result["predictions"]:
        print("Class: ", prediction['tagName'], ", Probability: ",
            float(prediction['probability'])*100, "%")
```

Le mode `'rb'` (lecture binaire) est indispensable pour une image ; `Content-Type: application/octet-stream` indique au serveur que le corps de la requête est constitué des octets de l'image, et `data=image_data` les envoie tels quels. La réponse JSON contient une liste `predictions` avec, pour chaque classe, un `tagName` et une `probability` comprise entre 0 et 1, que le programme affiche en pourcentage. (La version du programme reproduite ci-dessus est celle du déploiement Azure, d'où l'adresse IP ; pour le test local, `api_url` vaut `http://127.0.0.1/image`.)

# 2 Publier l'image sur Docker Hub

*Partie II du TP : l'image est envoyée sur un dépôt Docker Hub nommé `banana-classifier`, afin qu'Azure puisse la télécharger.*

Azure ne peut pas utiliser une image qui n'existe que sur la machine locale : il faut la placer dans un **registre**. Comme dans la Partie 1, on utilise Docker Hub, avec le nom de dépôt `banana-classifier` demandé par le sujet.

Listing 7 – Connexion, étiquetage et envoi de l'image

```
docker login
docker tag banana-classifier ygghest/banana-classifier
docker push ygghest/banana-classifier
```

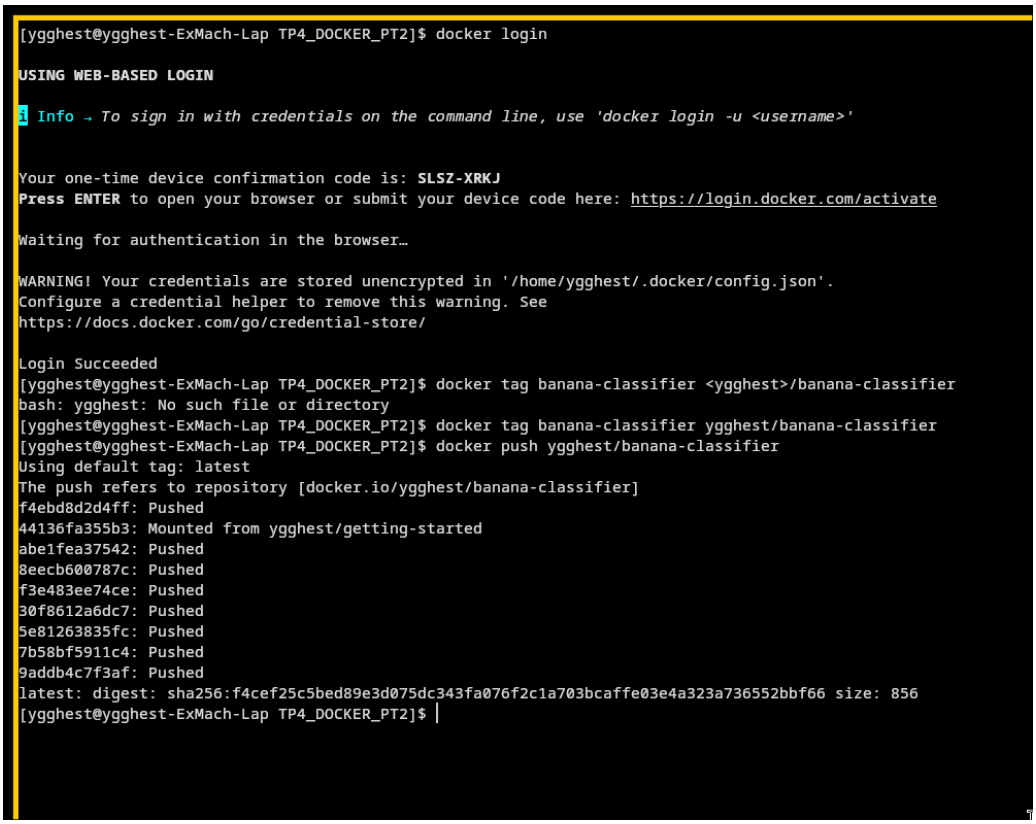



FIGURE 3 – Connexion (`Login Succeeded`), première tentative de `docker tag` en échec (`bash: ygghest: No such file or directory`), `docker tag` corrigé puis `docker push ygghest/banana-classifier` : le *digest* final est `sha256:f4cef25c5bed...`.

— **Connexion.** `docker login` utilise ici la connexion par navigateur : Docker affiche un code de confirmation à saisir sur `login.docker.com/activate`, puis indique `Login Succeeded`. L'avertissement signale que les identifiants sont stockés non chiffrés dans `~/.docker/config.json`.

— **Une première erreur de `docker tag`.** La première commande a été écrite avec `<ygghest>/banana-classifier`, en gardant les chevrons du modèle. Pour le shell, `<` est une redirection d'entrée : il a cherché un fichier `ygghest` et répondu `No such file or directory`. Les chevrons indiquent seulement où mettre son identifiant ; la commande correcte est `docker tag banana-classifier ygghest/banana-classifier`.

— **Étiquetage.** `docker tag SOURCE CIBLE` ajoute un second nom à l'image, de la forme `identifiant/dépôt`, sans la copier. C'est ce nom qui indique à `docker push` dans quel dépôt envoyer l'image.

— **Envoi.** Sans étiquette précisée, Docker utilise `latest` (`Using default tag: latest`). Docker n'envoie que les couches absentes du registre : l'une d'elles est signalée `Mounted from ygghest/getting-started`, c'est-à-dire qu'elle existait déjà dans un autre dépôt du même compte et a seulement été rattachée. Le *digest* `sha256:f4cef25c5bed...` est le même que celui du manifeste exporté à la fin de la construction (Figure 1), ce qui confirme que c'est bien cette image qui a été publiée.

# 3 Exécuter le conteneur dans Azure : Container Instance

*Partie III du TP : création du groupe de ressources, déploiement d'une Container Instance à partir de l'image Docker Hub, puis appel du service depuis Internet.*

## 3.1 Partie III.1 — Groupe de ressources

Un **groupe de ressources** est un conteneur logique dans lequel Azure range les ressources d'un même projet. Il se gère en bloc : supprimer le groupe supprime toutes les ressources qu'il contient, ce qui permet de tout nettoyer à la fin du TP. Le groupe `tp_docker` est créé dans une région disponible pour l'abonnement (*Spain Central*).




FIGURE 4 – Le groupe de ressources `tp_docker` apparaît dans la liste des ressources de l'abonnement.

## 3.2 Partie III.2 — Création de la Container Instance

Dans le portail, *Container Instances* puis *Create*, dans le groupe `tp_docker`, avec les paramètres du sujet :

TABLE 6: Paramètres de la Container Instance (les autres restent par défaut)

| **Paramètre** | **Valeur** | **Explication** |
|---|---|---|
| Name | `banana-classifier-instance` | Nom de la ressource |
| Source | *Other registry* | L'image ne vient pas du registre Azure mais d'un registre externe, Docker Hub |
| Image type | *Private* | Le registre demande une authentification : Azure doit donc recevoir des identifiants |
| Image | `ygghest/banana-classifier` | Dépôt Docker Hub contenant l'image |
| Registry login server | `index.docker.io` | Adresse du serveur de Docker Hub |
| Registry user name / password | Identifiants Docker Hub | Permettent à Azure de télécharger l'image |

| Paramètre | Valeur | Explication |
|---|---|---|
| Azure Spot | Décoché | Un conteneur *Spot* peut être arrêté à tout moment par Azure ; on veut un service stable |
| Operating System | Linux | L'image est une image Linux |
| Networking type | *Public* | Le conteneur reçoit une adresse IP publique |
| Ports | TCP 80 | Port sur lequel écoute le serveur Flask du conteneur |

Le sujet impose de montrer la configuration à l'enseignant avant de cliquer sur *Create*.

## 3.3 Partie III.3 — Déploiement

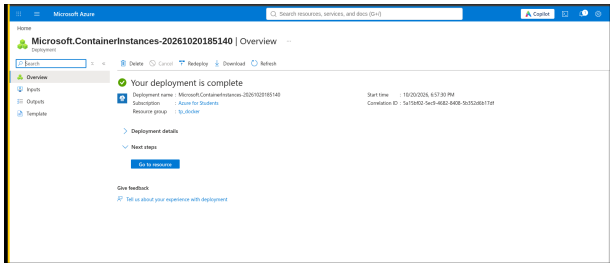



FIGURE 5 – *Your deployment is complete* : le déploiement `Microsoft.ContainerInstances-20261020185140` s'est terminé dans le groupe `tp_docker` (abonnement *Azure for Students*).

## 3.4 Partie III.4 — Adresse IP publique

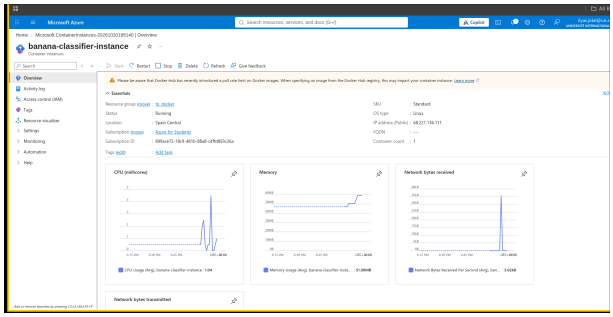



FIGURE 6 – Page de la ressource `banana-classifier-instance` : *Status* : `Running`, *Location* : `Spain Central`, *OS type* : `Linux`, *Container count* : 1 et *IP address (Public)* : `68.221.136.111`. Les graphiques en dessous montrent l'activité du conteneur (processeur, mémoire, octets reçus).

L'instance est en cours d'exécution et son adresse publique est `68.221.136.111`. Un bandeau d'avertissement rappelle que Docker Hub limite le nombre de téléchargements d'images (*pull rate limit*), ce qui peut gêner le démarrage d'instances qui récupèrent leur image sur ce registre. Les graphiques de métriques montrent, à l'heure de l'appel de l'API, un pic de processeur et d'octets reçus (l'image envoyée) ; la mémoire utilisée reste d'environ 52 Mo.

## 3.5 Partie III.5 — Appel de l'API déployée

Le programme de la Partie I est adapté en remplaçant l'adresse locale par celle de l'instance : `api_url = "http://68.221.136.111/image"`, sans autre changement (Figure 7).

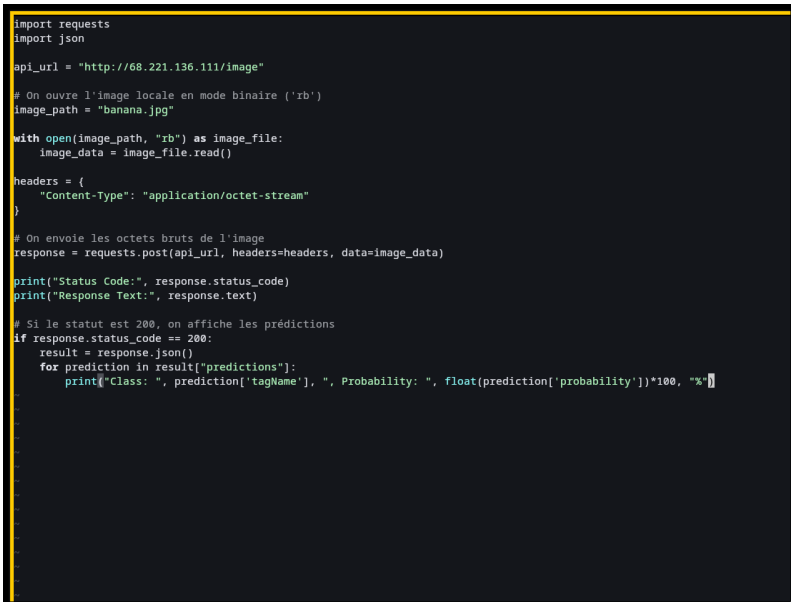



FIGURE 7 – Programme `test_banana.py` ouvert dans l'éditeur : `api_url` pointe vers l'adresse publique de la Container Instance.

Listing 8 – Exécution du programme

```
python3 test_banana.py
```

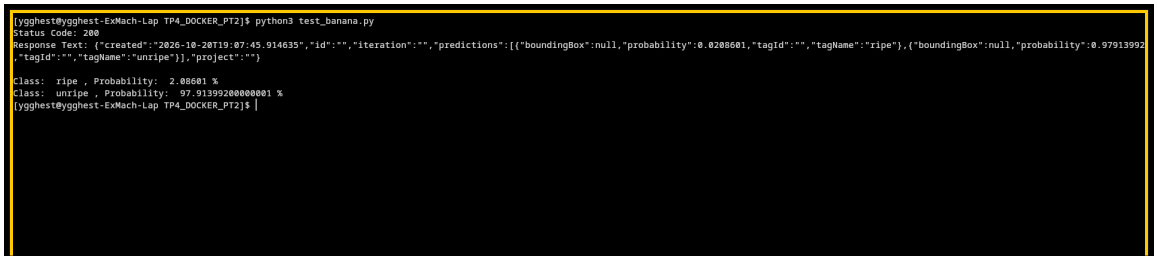



FIGURE 8 – Résultat de l'appel : `Status Code: 200`, puis la réponse JSON et la classification : `ripe` 2,086 % et `unripe` 97,914 %.

Le code de statut `200` indique que le conteneur déployé dans Azure a répondu. L'image envoyée est classée **pas mûre** (`unripe`) avec une probabilité d'environ 97,9 %. Le champ

`created` de la réponse est l'horodatage UTC calculé par le conteneur au moment de la prédiction.

# 4 Rendre le conteneur extensible : Container App

*Partie IV du TP : la même image est déployée comme Container App, qui fournit un point d'entrée HTTP et peut créer automatiquement de nouvelles instances.*

## 4.1 Partie IV.1 — Création de la Container App

Dans le portail, *Container Apps* puis *Create*, dans le groupe `tp_docker`, avec les paramètres du sujet :

TABLE 7: Paramètres de la Container App (les autres restent par défaut)

| **Paramètre** | **Valeur** | **Explication** |
|---|---|---|
| Name | `banana-classifier-app` | Nom de l'application |
| Region | *Spain Central* | Même région que le groupe de ressources |
| Use quickstart image | Décoché | On veut déployer notre propre image et non l'image d'exemple |
| Image source | *Docker Hub* | Registre d'où provient l'image |
| Image type | *Private* | Identifiants Docker Hub (nom d'utilisateur et mot de passe) à fournir |
| Image and tag | `ygghest/banana-classifier` | Dépôt Docker Hub de l'image (étiquette `latest`) |
| Ingress | *Enabled* | Active un point d'entrée HTTP géré par Azure qui répartit les requêtes entre les instances du conteneur |
| Ingress traffic | *Accepting traffic from anywhere* | L'application est accessible depuis Internet |
| Ingress type | HTTP | Trafic web |
| Target port | 80 | Port d'écoute du serveur Flask dans le conteneur |

Le sujet impose de montrer la configuration à l'enseignant avant de cliquer sur *Create*.

## 4.2 Partie IV.2 — Déploiement

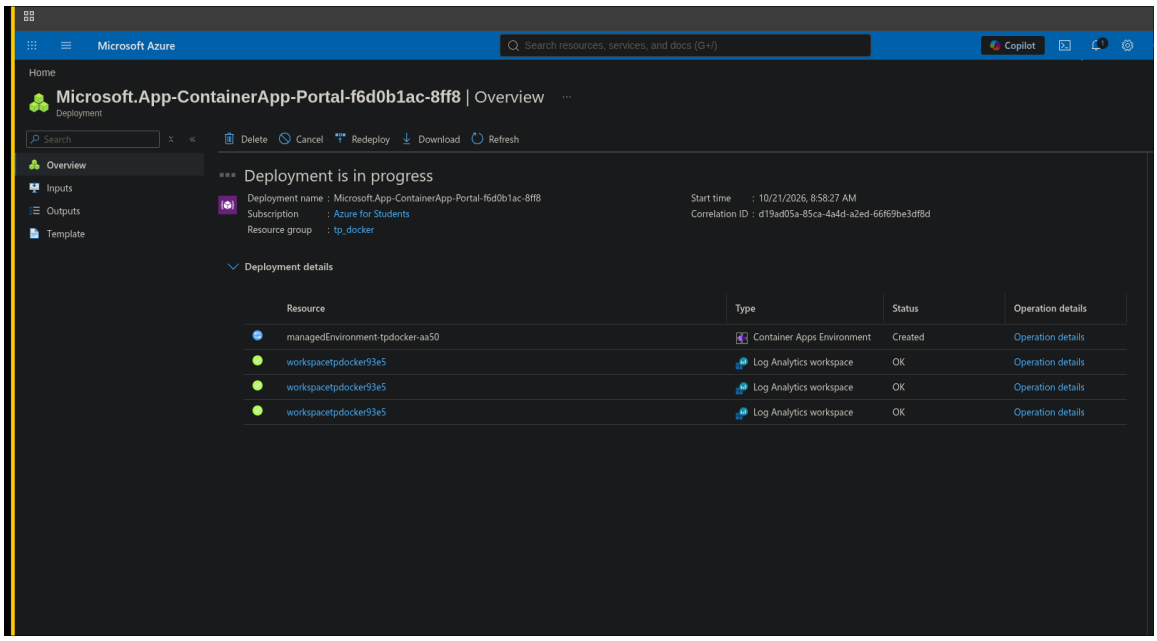



FIGURE 9 – Déploiement de la Container App en cours : Azure crée d'abord un *Container Apps Environment* (`managedEnvironment-tpdocker-aa50`) et un espace de travail *Log Analytics* (`workspacetpdocker93e5`), puis l'application elle-même.

Contrairement à une Container Instance, une Container App s'exécute dans un **environnement géré** (*Container Apps Environment*) qu'Azure crée automatiquement avec un espace de travail *Log Analytics* destiné aux journaux de l'application.

## 4.3 Partie IV.3 — URL de l'application

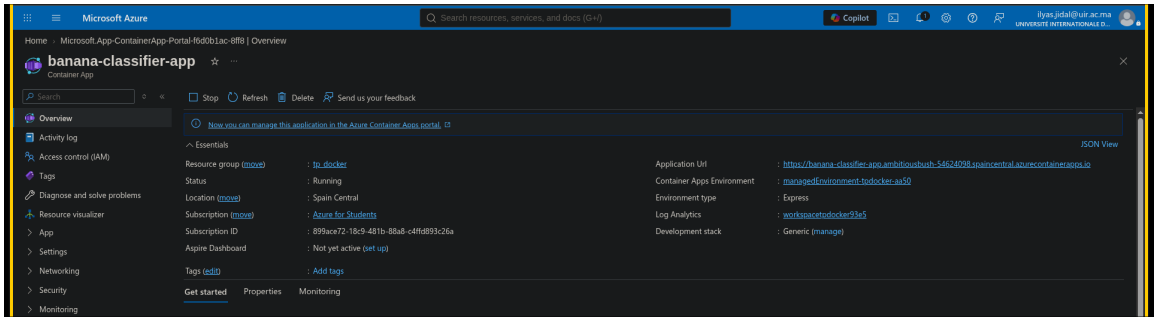



FIGURE 10 – Page de la ressource `banana-classifier-app` : *Status* : `Running`, *Location* : `Spain Central`, et *Application Url* : `https://banana-classifier-app.ambitiousbush-54624098.spaincentral.azurecontainerapps.io`.

L'adresse fournie est un nom de domaine en **HTTPS**, géré par Azure, alors que le conteneur lui-même n'écoute qu'en HTTP sur le port 80 : c'est l'*ingress* qui reçoit les requêtes et les transmet au conteneur sur le *Target port*.

## 4.4 Partie IV.4 — Appel de l'API

Le programme est de nouveau adapté en remplaçant `api_url` par l'adresse de l'application, en conservant la route `/image` :

Listing 9 – Nouvelle valeur de `api_url`

```
api_url = "https://banana-classifier-app.ambitiousbush-54624098.
    spaincentral.azurecontainerapps.io/image"
```

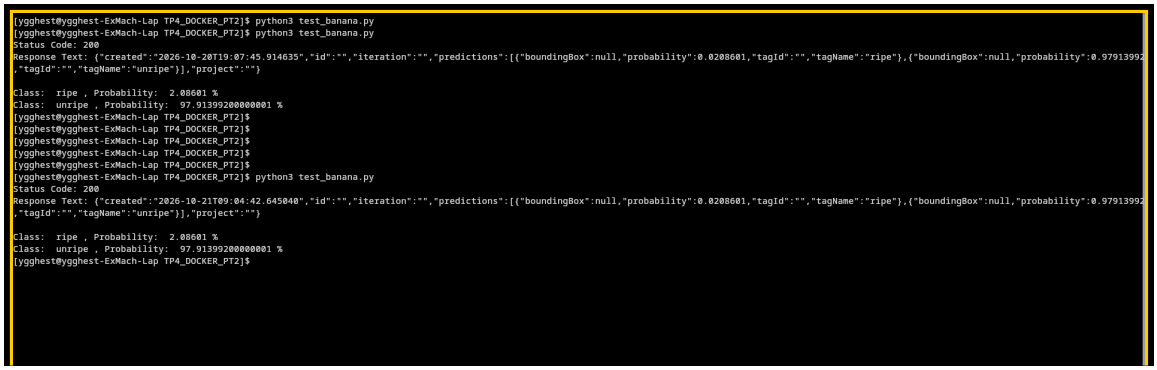



FIGURE 11 – Nouvelle exécution de `python3 test_banana.py` (second bloc de sortie, en bas) : `Status Code: 200`, avec le même résultat, `ripe` 2,086 % et `unripe` 97,914 %. Le premier bloc, au-dessus, est la sortie du test précédent sur la Container Instance.

Le service répond avec le même classement que sur la Container Instance, puisqu'il s'agit de la même image et de la même image de test. Seul l'horodatage `created` change (`2026-10-20T19:07:45` pour la première exécution, `2026-10-21T09:04:42` pour la seconde), ce qui montre que la seconde réponse vient bien d'un nouvel appel, cette fois à la Container App.

# 5 Modifier la règle de mise à l'échelle

*Partie V du TP : examen puis remplacement de la règle `http-scaler` de la Container App.*

## 5.1 Partie V.1 — Principe

Dans *Application → Scale*, Azure applique par défaut une règle `http-scaler` : l'application crée **une nouvelle instance (réplica) du conteneur toutes les 10 requêtes HTTP simultanées**. Le nombre de réplicas varie donc automatiquement avec le trafic reçu, entre un minimum et un maximum.

## 5.2 Partie V.2 — Nouvelle règle

La règle par défaut est supprimée et remplacée par la suivante, pour qu'une nouvelle instance soit créée **pour chaque requête simultanée**, avec **au plus 5 instances** :

| Paramètre | Valeur | Explication |
|---|---|---|
| Max replicas | 5 | Nombre maximal d'instances du conteneur |
| Rule name | `my-http-scaler` | Nom de la nouvelle règle |
| Type | *HTTP scaling* | La mise à l'échelle dépend du nombre de requêtes HTTP |
| Concurrent requests | 1 | Seuil de requêtes simultanées par instance : au-delà de 1, une instance supplémentaire est créée |

TABLE 8: Nouvelle règle de mise à l'échelle

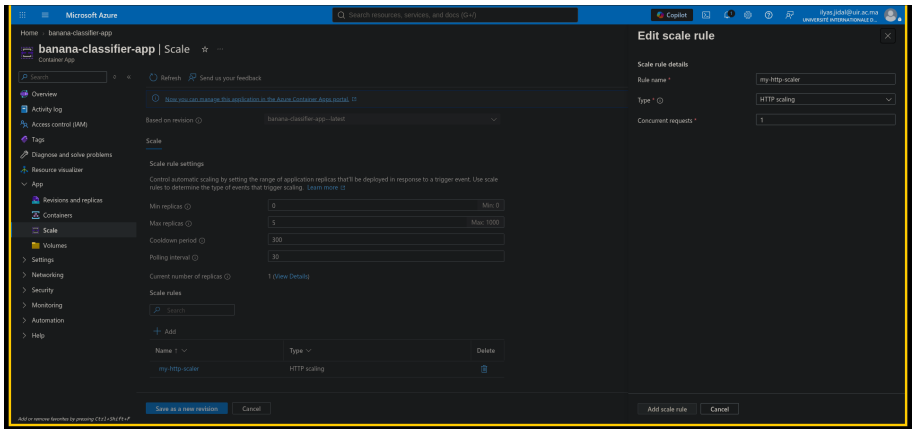



FIGURE 12 – Page *Scale* de `banana-classifier-app` : *Max replicas* = 5, règle `my-http-scaler` de type *HTTP scaling*, et panneau *Edit scale rule* avec *Concurrent requests* = 1. Le bouton *Save as a new revision* enregistre la modification.

## 5.3 Partie V.3 — Lecture des paramètres

- **Min replicas = 0** : lorsqu'aucune requête n'arrive, l'application peut descendre à *zéro* instance et ne consomme alors plus de ressources de calcul ; la première requête qui suit doit redémarrer une instance, ce qui peut allonger légèrement sa durée de réponse.
- **Max replicas = 5** : plafond du nombre d'instances, qui limite le coût en cas de forte charge.
- **Cooldown period = 300** (secondes) : délai d'attente après la dernière requête avant de réduire le nombre d'instances.
- **Polling interval = 30** (secondes) : fréquence à laquelle Azure vérifie le niveau de trafic pour décider d'ajouter ou de retirer des instances.
- **Current number of replicas = 1** au moment de la capture : une seule instance suffit tant que le trafic reste faible.

Modifier la règle d'échelle crée une **nouvelle révision** de l'application (le bouton affiche *Save as a new revision*). Une révision est une version figée de la configuration de l'application ; la nouvelle révision active applique les règles `my-http-scaler` et le maximum de 5 réplicas.

# 6 Devoir : exécuter un conteneur dans Azure

## 6.1 Les étapes

1. **Obtenir l'image.** Construire l'image à partir du Dockerfile (`docker build -t banana-classifier .`) et la tester en local (`docker run -d -rm -p 80:80 banana-classifier`, puis appel de l'API).
2. **La publier dans un registre accessible depuis Azure.** `docker login`, `docker tag banana-classifier ygghest/banana-classifier`, `docker push ygghest/banana-classifier`.
3. **Créer un groupe de ressources** (`tp_docker`) qui rassemblera toutes les ressources du TP.
4. **Choisir le service de déploiement et le configurer** : source de l'image (registre externe), identifiants si le dépôt est privé, système d'exploitation, port à exposer et réseau (adresse IP publique pour une Container Instance, *ingress* pour une Container App).
5. **Déployer, puis récupérer le point d'entrée** : l'adresse IP publique de l'instance ou l'URL de l'application.
6. **Appeler le service** avec un programme ou `curl`, en remplaçant simplement l'adresse dans `api_url`.
7. **Régler le comportement** (Container App) : règles de mise à l'échelle et nombre maximal de réplicas.
8. **Supprimer le groupe de ressources** à la fin du TP, pour arrêter la facturation et ne laisser aucune ressource exposée sur Internet.

| **Commande ou option** | **Rôle** |
|---|---|
| `docker build -t banana-classifier .` | Construit l'image à partir du Dockerfile du dossier courant et la nomme |
| `docker run -d -rm -p 80:80` | Lance le conteneur en arrière-plan (`-d`), le supprime à l'arrêt (`-rm`) et relie le port 80 de l'hôte à celui du conteneur (`-p hôte:conteneur`) |
| `docker login` | Authentifie le client Docker auprès de Docker Hub |
| `docker tag SOURCE CIBLE` | Donne à l'image le nom `identifiant/dépôt`, nécessaire pour l'envoyer dans son dépôt |
| `docker push ygghest/banana-classifier` | Envoie les couches de l'image vers Docker Hub |
| `requests.post(url, headers=..., data=...)` | Envoie une requête HTTP `POST` avec un en-tête `Content-Type` et les octets de l'image |
| *Registry login server* `index.docker.io` | Serveur du registre où Azure va chercher l'image |
| *Target port* 80 / port TCP 80 | Port d'écoute du conteneur vers lequel Azure dirige le trafic |
| *Concurrent requests* / *Max replicas* | Seuil de requêtes simultanées par instance / nombre maximal d'instances d'une Container App |

| Commande ou option | Rôle |
|---|---|

TABLE 9: Commandes et options utilisées dans le TP

## 6.2 Container Instance ou Container App

| Critère | Azure Container Instance | Azure Container App |
|---|---|---|
| Principe | Exécute directement un conteneur (ou un groupe de conteneurs), sans serveur à gérer : c'est la façon la plus simple de lancer un conteneur dans Azure | Plateforme d'applications conteneurisées, gérée par Azure et construite sur Kubernetes (que l'utilisateur n'a pas à administrer) |
| Mise à l'échelle | Aucune automatique : une instance de taille fixe ; pour en avoir davantage, il faut en créer d'autres soi-même | Automatique, selon des règles (requêtes HTTP, événements, processeur...), entre un minimum (possible : 0) et un maximum de réplicas |
| Accès réseau | Adresse IP publique directe et port ouvert ; ici en HTTP sur `68.221.136.111` | *Ingress* géré avec un nom de domaine en HTTPS et un répartiteur de charge entre les réplicas ; ici l'URL en `azurecontainerapps.io` |
| Mises à jour | Pas de notion de version : on redéploie le conteneur | **Révisions** : chaque changement de configuration crée une nouvelle révision |
| Environnement | Ressource autonome | Rattachée à un *Container Apps Environment* avec un espace de travail *Log Analytics* pour les journaux |
| Coût | Facturé selon les ressources (processeur, mémoire) tant que l'instance s'exécute | À l'usage : ne consomme plus de calcul lorsque l'application est descendue à zéro réplica |
| Usage typique | Tests rapides, tâches ponctuelles ou par lots, petit service isolé | Applications web, API et microservices à charge variable |

TABLE 10: Différences entre Azure Container Instance et Azure Container App

En résumé, la **Container Instance** sert à lancer vite un conteneur qui tourne tel quel, tandis que la **Container App** ajoute ce qu'il faut pour exploiter un service : point d'entrée HTTPS, répartition de charge, révisions et adaptation automatique à la charge. Dans ce TP, les deux ont servi la même image et renvoyé le même résultat ; la différence tient à la manière dont le service est exposé et dimensionné.

## 6.3 Suppression des ressources

Le sujet demande de supprimer le groupe de ressources à la fin du TP. Cette suppression retire d'un coup l'instance, l'application, l'environnement et l'espace de travail *Log Analytics* qui lui sont rattachés ; elle se fait depuis le portail (page du groupe `tp_docker`, bouton *Delete resource group*) ou avec Azure CLI :

Listing 10 – Suppression du groupe de ressources avec Azure CLI

```
1 az group delete --name tp_docker --yes
```

# 7 Bilan des éléments créés

| Élément | Rôle |
|---|---|
| Image `banana-classifier` | Classifieur ripe / unripe : Flask, TensorFlow Lite et modèle exporté de Custom Vision |
| Dépôt Docker Hub `ygghest/banana-classifier` | Registre d'où Azure télécharge l'image |
| Groupe de ressources `tp_docker` | Regroupe les ressources Azure du TP (région *Spain Central*) |
| Container Instance `banana-classifier-instance` | Conteneur unique, adresse IP publique `68.221.136.111`, port 80 |
| Container App `banana-classifier-app` | Même image, avec *ingress* HTTPS et mise à l'échelle automatique |
| *Container Apps Environment* `managedEnvironment-tpdocker-aa50` | Environnement géré créé avec la Container App |
| Espace de travail *Log Analytics* `workspacetpdocker93e5` | Collecte les journaux de la Container App |
| Règle `my-http-scaler` | Une instance par requête simultanée, 5 instances au maximum |
| Programme `test_banana.py` | Appelle l'API du classifieur (local, Container Instance, Container App) |

TABLE 11: Éléments créés pendant le TP4 Partie 2

# 8 Difficultés rencontrées

| Problème | Cause | Solution |
|---|---|---|
| `docker tag banana-classifier <ygghest>/banana-classifier` : *bash : ygghest : No such file or directory* | Les chevrons du modèle ont été gardés ; le shell interprète `<` comme une redirection d'entrée | Écrire l'identifiant sans chevrons : `docker tag banana-classifier ygghest/banana-classifier` |
| Avertissement *JSONArgsRecommended* à la construction | `CMD` écrite en forme shell dans le Dockerfile exporté | Sans conséquence ici ; on peut passer à la forme JSON `CMD ["python", "-u", "app.py"]` |
| Programme d'appel du sujet fondé sur une URL d'image (`/url`) | Le conteneur télécharge lui-même l'image, ce qui exige qu'il ait accès à Internet et que l'adresse soit valide | Envoyer l'image locale `banana.jpg` en octets bruts vers `/image` (`Content-Type: application/octet-stream`) |
| Avertissement de Docker Hub sur la limite de téléchargements (*pull rate limit*) dans la page de la Container Instance | Docker Hub limite le nombre de téléchargements d'images, en particulier sans authentification | Fournir les identifiants Docker Hub lors du déploiement, comme le demande le sujet |

TABLE 12: Problèmes rencontrés et solutions